

AI GitHub Repository Health Monitor

Project Requirements Specification

Version 1.0.0

1. Project Overview

Build a production-oriented **AI GitHub Repository Health Monitor** that automatically evaluates the health of GitHub repositories and provides developers with a clear health score, detected problems, explanations, recommendations, historical trends, and notifications.

The product must start as a **personal developer tool** but its architecture must be **SaaS-ready from day one**. The system must support both public and private GitHub repositories through a secure GitHub App integration.

The platform should minimize manual monitoring by combining GitHub events, scheduled repository scans, deterministic analysis, AI-assisted analysis, health scoring, and notifications.

The system must prioritize reliability, security, explainability, extensibility, and maintainability.

2. Primary Objectives

The system must:

1. Connect a user's GitHub account through a GitHub App.
2. Discover repositories the connected installation is authorized to access.
3. Allow users to select repositories for monitoring.
4. Perform an initial repository health scan.
5. Perform incremental analysis when relevant GitHub events occur.
6. Perform scheduled full repository scans.
7. Analyze multiple repository-health categories.
8. Calculate an overall health score from 0–100.
9. Allow health-category weights to be configured.
10. Explain why a score changed.
11. Use AI to generate contextual recommendations.

12. Maintain historical health scores and findings.
 13. Notify users about important health changes.
 14. Allow users to approve certain AI-suggested actions.
 15. Maintain an auditable record of automated and user-approved actions.
 16. Protect repository credentials, source information, and user data.
 17. Support future integrations without major architectural changes.
-

3. Target Users

Primary User

Developers who maintain one or more GitHub repositories and want automated visibility into repository quality, security, maintainability, activity, and technical debt.

Future Users

- Development teams
- Freelancers
- Startups
- Engineering managers
- Agencies
- Organizations managing many repositories

The architecture must therefore support future multi-user and multi-tenant operation.

4. Core Functional Requirements

4.1 Authentication and GitHub Connection

The application must provide secure user authentication and a GitHub App installation flow.

After installation, the system must:

- Identify the connected GitHub account.
- Retrieve authorized repositories.

- Respect GitHub repository permissions.
- Never request unnecessary repository permissions.
- Store only required GitHub integration metadata.
- Support installation removal/revocation.

The system must never require users to provide raw GitHub Personal Access Tokens for normal operation.

4.2 Repository Management

Users must be able to:

- View available repositories.
- Add repositories to monitoring.
- Remove repositories from monitoring.
- View repository metadata.
- Trigger a manual scan.
- View current health status.
- View historical health information.

The system must distinguish between repositories that are merely accessible and repositories actively monitored.

4.3 Repository Health Analysis

The initial system must evaluate at least these categories:

- Security
- Code Quality
- Dependencies
- Documentation
- Issues
- Pull Requests
- Repository Activity
- Repository Configuration

Each category must produce:

- Category score
- Findings
- Severity
- Evidence
- Explanation
- Recommendations

The analysis must distinguish between deterministic findings and AI-generated interpretations.

4.4 Health Score

Every monitored repository must receive an overall score from **0 to 100**.

The score must be calculated from configurable category weights.

Example:

Security 20%

Code Quality 20%

Dependencies 15%

Documentation 10%

Issues 10%

Pull Requests 10%

Activity 10%

Configuration 5%

Total 100%

The default weights must be configurable without requiring code changes.

The system must preserve the scoring configuration used for each historical scan so historical scores remain reproducible and explainable.

4.5 Findings

Findings must contain sufficient information to understand the problem.

Each finding should include:

- Category
- Severity
- Title
- Description
- Evidence
- Detection source
- Affected repository/resource
- First detected timestamp
- Last detected timestamp
- Current status
- Recommendation

Severity should support at least:

Critical

High

Medium

Low

Informational

4.6 AI Analysis

AI must not replace deterministic analysis where deterministic rules are available.

Use traditional programmatic analysis for measurable signals such as:

- Repository activity
- Dependency age
- Issue age
- PR age

- Missing files
- Configuration checks
- Repository metadata
- Available GitHub security information

Use AI primarily for:

- Explaining findings
- Prioritizing recommendations
- Detecting contextual risks
- Summarizing repository health
- Generating actionable recommendations
- Explaining score changes
- Producing developer-friendly reports

The AI layer must be provider-agnostic.

No provider-specific implementation should leak into the core business logic.

4.7 Notifications

V1 must support:

- In-app notifications
- Email notifications

Notifications should be generated for important events such as:

- Critical/high-severity findings
- Significant health-score degradation
- New security-related findings
- Failed scans
- Repository monitoring failures
- Completed important AI recommendations
- Approved automation actions

Slack and Discord must be considered future integrations and should not be tightly coupled to the notification architecture.

4.8 AI-Assisted Actions

AI may recommend actions but must not make potentially destructive repository changes autonomously.

Supported future/V1-safe actions may include:

- Create GitHub issue
- Create GitHub pull request
- Generate documentation update
- Generate configuration recommendation

Actions that modify repositories must require explicit user approval unless a future permission system explicitly enables otherwise.

Every action must be logged for auditing.

5. Automation Requirements

The platform must use two complementary mechanisms.

Event-Based Monitoring

GitHub webhooks should trigger targeted processing for relevant events such as:

- Push
- Pull request changes
- Issues
- Releases
- Repository configuration changes
- Security-related events where supported

The event handler should enqueue work rather than performing heavy processing synchronously.

Scheduled Monitoring

A scheduled full scan must periodically reevaluate monitored repositories.

Scheduled scanning is required because not every important health change will necessarily arrive through a webhook.

The scanning system must support:

- Retry
 - Failure tracking
 - Idempotency
 - Job status
 - Timeouts
 - Rate-limit awareness
-

6. Dashboard Requirements

The dashboard must provide:

Overview

- Overall health score
- Score trend
- Category scores
- Critical findings
- Recent scan
- Repository status

Repository Detail

- Current score
- Category breakdown
- Findings
- Recommendations
- Score history

- Scan history
- Recent repository activity
- AI health summary

Finding Detail

- Problem
- Severity
- Evidence
- Explanation
- Recommendation
- Detection history
- Related repository resource
- Available action

The UI must clearly distinguish facts detected from GitHub/programmatic analysis from AI-generated recommendations.

(Priority-High) The UI of the dashboard must be component-based design with drag and drop features so that user can easily shift the position of the component of dashboard as per there need.

7. Non-Functional Requirements

Security

- Follow least-privilege access.
- Encrypt sensitive data where appropriate.
- Never expose GitHub credentials to the frontend.
- Validate webhook signatures.
- Apply authentication and authorization to all protected APIs.
- Isolate tenant/user data.
- Maintain audit logs for security-sensitive actions.

Performance

The API must remain responsive while scans execute asynchronously.

Long-running scans must never block normal API requests.

Reliability

The system must tolerate:

- GitHub API failures
- Temporary network failures
- AI provider failures
- Worker failures
- Database failures where recoverable

Failed jobs must be observable and retryable.

Scalability

The architecture must allow horizontal scaling of:

- API servers
- Background workers
- Scan workers
- AI processing workers

The system must avoid designs that require a single global worker or process.

Maintainability

Use clear separation between:

API

Business Logic

GitHub Integration

Scanning Engine

Scoring Engine

AI Layer

Notification Layer

Persistence

Background Jobs

8. Technology Baseline

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS

Backend

- Python
- FastAPI
- Pydantic
- SQLAlchemy

Database

- PostgreSQL

Background Processing

- Redis
- Background job/worker system

AI

- Provider-agnostic LLM abstraction

Integration

- GitHub App
- GitHub REST/GraphQL APIs
- GitHub Webhooks

Infrastructure

- Docker
- GitHub Actions
- Vercel for frontend where appropriate
- Render/Railway or equivalent for backend initially

The architecture must remain deployable to other cloud providers later.

9. SaaS Readiness

Although the first deployment may be used by one developer, all core resources must be designed with ownership boundaries.

The data model must be capable of supporting:

User



Workspace / Tenant



GitHub Installation



Repositories



Scans / Findings / Scores / Notifications

Do not build a single-user architecture that requires major database or authorization redesign when multi-user SaaS functionality is introduced.

10. Success Criteria

The first production-capable version is successful when a user can:

1. Sign in.
2. Install/connect the GitHub App.

3. Select a public or private repository.
4. Start an initial scan.
5. Receive a 0–100 health score.
6. View category scores.
7. View actionable findings.
8. Receive AI-generated explanations and recommendations.
9. See historical score changes.
10. Receive important notifications.
11. Trigger scans manually.
12. Have repository changes trigger appropriate automated analysis.
13. Safely approve an AI-suggested repository action.
14. Audit what the system detected and what action was taken.

All implementation decisions in later phases must remain consistent with these requirements.